Predator/Prey Simulation - UML Class Diagram

Strategy pattern participants

Other classes

Entity

-x: int
-y: int
-isFood: boolean

+getX(): int / +getY(): int
+setX(int) / +setY(int)
+isFood(): boolean

«abstract»
Creature

-speed: int
-starvation: int
-lastX, -lastY, -dx, -dy: int
-strategy: MovementStrategy
-alive: boolean
+WORLD_WIDTH, +WORLD_HEIGHT: int {static final}
#SIZE: int {static final}
-BASE_SPEED, -MIN_SPEED, -MAX_SPEED: int; -METABOLIC_STEP: double {static final}

+starve(): boolean
#getStarvationRate(): int
-metabolicAdjustment(): int
#mutatedSpeed(): int
#chooseStrategy()
+movement()
#applyStrategy()
#setStrategy(MovementStrategy)
#moveWithBounce()
#clampToWorld()
#recordPosition()
+eat()
+reproduce()
+distanceTo(Entity): double
+findClosest(List<E>): Optional<E>
+findClosest(Stream<E>): Optional<E>
#findNearestInRange(List<E>, int): Optional<E>
#ofType(List, Class<T>): List<T> {static}
+getSpeed(): int / +setSpeed(int)
+getStarvation(): int / +resetStarvation()
+isAlive(): boolean / +kill()
+getVelocityX(): int / +getVelocityY(): int

strategy

«interface»
«functional»
MovementStrategy

+move(self: Creature)

returns lambdas implementing

«final utility class»
MovementStrategies

-MovementStrategies()
+wander(): MovementStrategy
+idle(): MovementStrategy
+seek(Entity): MovementStrategy
+flee(Creature, double): MovementStrategy
+pursue(Creature, double): MovementStrategy
+herd(List<? extends Creature>, int): MovementStrategy

Not drawn, to keep lines readable: Prey depends on Predator (flees) and Grass (grazes); Predator depends on Prey (hunts); Simulation creates Predator, Prey and Grass.

Grass

-growthTimer: int
-maxGrowthTimer: int {final}
-isEaten: boolean

+isEdible(): boolean
+getGrowthTimer(): int
+setGrowthTimer(int)
+isEaten(): boolean
+setEaten(boolean)
+tickGrowth()

*grassList **

Simulation

~sim: List<Creature>
~grassList: List<Grass>
-random: Random
-frameCounter: int
-gameTimer: Timer
-simPanel: SimPanel
-graph: PopulationGraph
-start-screen Swing fields (JFrame, JPanel, JButton, JLabel, JTextField)
-RENDER_INTERVAL_MS, TICKS_PER_LOGIC_UPDATE, GRASS_SIZE, GRASS_GROWTH_TIME, MAX_STARTING_POPULATION, GRAPH_HEIGHT: int {static final}

+main(String[]) {static}
+frameInitialise()
-parseAmount(String, String): int
-spawnGrass()
-startGameLoop()
+runSimulationTick()
+repaint()

*sim **

*sim **

uses

collects

Predator

-sim: List<Creature>
-random: Random {static}
-DETECTION_RANGE, EAT_RADIUS: int {static final}
-REPRODUCE_THRESHOLD, HUNT_THRESHOLD: int {static final}
-REPRODUCE_CHANCE, HUNT_SPEED_MULTIPLIER: double {static final}

#getStarvationRate(): int
+eat()
+reproduce()
#chooseStrategy()

Prey

-inDanger: boolean
-sim: List<Creature>
-grassList: List<Grass>
-starvationRate: int
-THREAT_RANGE, EAT_RADIUS, GRAZE_THRESHOLD: int {static final}
-HERD_RANGE, HERD_COMFORT, REPRODUCE_THRESHOLD: int {static final}
-FLEE_SPEED_MULTIPLIER, REPRODUCE_CHANCE: double {static final}

+isInDanger(): boolean
+setInDanger(boolean)
#getStarvationRate(): int
+eat()
+reproduce()
#chooseStrategy()

throws

graph

«inner class of Simulation»
extends JPanel
SimPanel

#paintComponent(Graphics)

extends JPanel
PopulationGraph

-sim: List<Creature>
-samples: Deque<double[]>
-previous: Set<Creature>
-currentPrey, -currentPredators: int
-peakPrey, -peakPredators, -peakTotal: int
-preyDeaths, -predatorDeaths: int
-preyStats, -predatorStats: SpeciesStats
-MAX_SAMPLES: int {static final}

+sample()
#paintComponent(Graphics)
-drawSpeedSegment(Graphics, Color, int, int, double, double, int)
-format(SpeciesStats): String {static}

stats

SpeciesStats

-count: int
-fastest: int
-averageSpeed: double

+empty(): SpeciesStats {static}
+count(): int
+fastest(): int
+averageSpeed(): double
+collector(): Collector<Creature, long[], SpeciesStats> {static}

extends Exception
InvalidPopulationException

+InvalidPopulationException(String)